

Improving Vision-Language-Action Model with Online Reinforcement Learning

Yanjiang Guo^{13*}, Jianke Zhang^{1*}, Xiaoyu Chen^{13*}, Xiang Ji¹, Yen-Jen Wang², Yucheng Hu¹, Jianyu Chen^{13†}

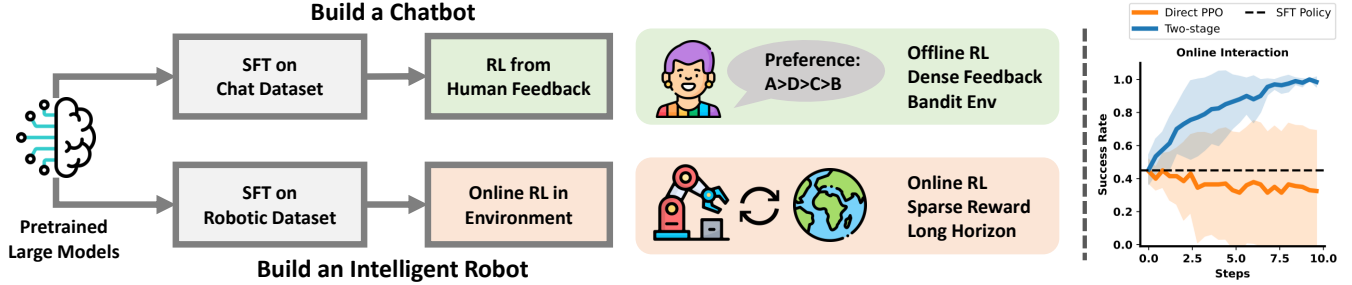


Fig. 1: Illustration of our motivation. We employ the fine-tuning pipeline from large language models (LLMs) to enhance the Vision-Language Architecture (VLA) in the robotic domain, starting with supervised fine-tuning (SFT) followed by reinforcement learning (RL). However, we observed that standard online RL can be extremely unstable when applied to large VLA models. To address this, we propose an iterative RL method, iRe-VLA.

Abstract—Recent studies have successfully integrated large vision-language models (VLMs) into low-level robotic control by supervised fine-tuning (SFT) with expert robotic datasets, resulting in what we term vision-language-action (VLA) models. Although the VLA models are powerful, how to improve these large models during interaction with environments remains an open question. In this paper, we explore how to further improve these VLA models via Reinforcement Learning (RL), a commonly used fine-tuning technique for large models. However, we find that directly applying online RL to large VLA models presents significant challenges, including training instability that severely impacts the performance of large models, and computing burdens that exceed the capabilities of most local machines. To address these challenges, we propose iRe-VLA framework, which iterates between Reinforcement Learning and Supervised Learning to effectively improve VLA models, leveraging the exploratory benefits of RL while maintaining the stability of supervised learning. Experiments in two simulated benchmarks and a real-world manipulation suite validate the effectiveness of our method.

I. INTRODUCTION

It has become a recent trend to employ powerful pre-trained large language models (LLMs) and vision-language models (VLMs) for a variety of advanced tasks beyond their original scope, including dialogue systems [1], [2], [3], code generation [4], task planning [5], [6], and even low-level robotic control [7], [8]. By fine-tuning VLMs on robotic datasets with explicit action modeling, previous works have developed large vision-language-action (VLA) models [9], such as RT-2 [8], HiRT[10], Roboflamingo [11], etc. These models are capable of directly outputting low-level robotic

control signals while also benefiting from the common-sense knowledge and reasoning abilities [12] encoded in large pre-trained models.

The fine-tuning of VLA models generally employs a supervised fine-tuning (SFT) approach [8], noted for its stability and scalability. However, SFT depends on high-quality expert datasets that are costly and difficult to obtain in the robotic domain [13]. Additionally, supervised learning may not fully align VLA models with physical environments due to distribution shift issues [14], [15]. We wonder how to further improve such large VLA models through interaction with the physical environment beyond supervised learning. Notably, Reinforcement Learning from Human Feedback (RLHF) [1], [16], [17] has better align large language model with human preference, as illustrated in the upper-left of Figure 1.

Inspired by the success of RLHF, we try online RL to improve the VLA model and better align the VLA model with physical environments. However, the environments encountered by chatbots and embodied robots are markedly different. Chatbots are optimized using offline, human-labeled datasets with well-defined dynamics [1], while embodied robots necessitate online exploration in tasks characterized by long horizons and sparse rewards. Furthermore, previous research has shown that the **online reinforcement learning (RL) process can be extremely unstable when applied to large neural networks**[18], [19], [20]. Empirically, we also observe that directly applying the standard RL algorithm to large VLA models results in training instability and performance drops, as depicted on right side of Figure 1.

To stabilize the RL process and effectively enhance the VLA model, we propose the novel **iRe-VLA** method, which iterates between online Reinforcement Learning stages and supervised learning stages. Specifically, during the RL stage,

*Equal contribution

†Corresponding author. jianyuchen@tsinghua.edu.cn

¹Institute for Interdisciplinary Information Sciences, Tsinghua University, Beijing, China. guoyj22@mails.tsinghua.edu.cn

²University of California, Berkeley, USA.

³Shanghai Qi Zhi Institute, Shanghai, China.

we freeze the VLM parameters and only train lightweight action heads to maintain training stability. In the subsequent supervised learning phase, we fine-tune the entire model on successful trajectories to fully utilize the expressive capabilities of the large model. Empirically, this two-stage approach consistently enhances the VLA’s performance, stabilizes training, and is computationally more efficient. We have validated the iRe-VLA methods through comprehensive experiments, including simulated MetaWorld [21], Franka-Kitchen [22], and real-world Panda manipulation task sets. In these domains, our method not only better aligns the VLA model with the original tasks but also autonomously solves unseen tasks. Furthermore, the VLA model’s generalization ability has also been improved through online interactions with the environment.

II. RELATED WORKS

Foundation Models for Embodied control. Large Language Models (LLMs) and vision-language models (VLMs) trained on web-scale data encode knowledge of the physical world and exhibit impressive reasoning ability. With this prior knowledge, LLMs and VLMs can benefit the embodied control tasks in many aspects, ranging from providing rewards or values [23], [24], [25] for agents, modeling the world dynamics [26], [27], or directly as policy [5], [6], [28], [29], [30], [31], [32].

As for literature using LLMs/VLMs directly as agents’ policy, we can roughly divide them into two categories, namely high-level planning and low-level control. Works in the first categories leverage LLMs’ reasoning ability to autoregressively generate the textual step sequences [5], [6], [28] or code [33], thereby decomposing the long-horizon tasks into feasible plans. However, these methods output textual plans that are not directly grounded in the physical world and require powerful low-level skills. Another line of work leveraged VLMs to directly output low-level control signals and verified that low-level skills themselves could also benefit from the prior knowledge encoded in the pre-trained VLMs [7], [8], [10], [34], [11]. Since the original output of VLMs lies in the language space, these works need additional action modeling parts like adding action heads [10], [11] or replacing the language tokens with actions [8].

Finetune Large Models with RL. Reinforcement learning has been successfully used in the natural language process downstream tasks to better align the generated text to human preferences [1], [35], [36]. In this Reinforcement Learning from Human Feedback (RLHF) framework, a reward model is trained on a pre-collected human preference dataset and then LLM is optimized in a bandit environment with constraints of not shifting too much from the original model [1], which can be seen as offline-style RL [37]. Different from RLHF for dialog systems, fine-tuning VLA models face unknown dynamics and require online exploration [38], [39], [40]. For instance, GLAM [38] ground the LLM textual plans in simplified grid-world environments through online RL. LLaRP [39] ground the high-level plans generated by VLMs in rearrangement tasks with dense reward RL.

However, they all assume low-level skills (e.g., pick, goto) are available and only better ground the high-level plans. Different from them, we try to use RL to directly improve the low-level control signal output by VLA policy which has much longer horizons (hundreds or thousands of steps) in sparse-reward physical environments.

III. PRELIMINARY

Reinforcement Learning. We utilize the standard deep RL partially-observed Markov decision process (POMDP) framework, where a task can be modeled as $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P_T, R, \gamma, \mathcal{O}, P_E)$. $\mathcal{S}$ and $\mathcal{A}$ are the state space and action space for tasks, $\mathcal{O}$ is the robot observation, such as visual image. $P_T : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ are state transition probability functions and $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ are reward function for the task. In robotic tasks, the reward signal is always sparse, so we consider binary reward in this paper, where $R = 1$ if the robot successfully finished the task otherwise $R = 0$. $P_E : \mathcal{S} \times \mathcal{O} \rightarrow [0, 1]$ is the observation emission probabilities. A policy $\pi_\theta : \mathcal{O} \rightarrow \mathcal{A}$ defines a probability distribution in action space parameterized by θ . The objective of parameter θ is to maximize the expected return of the policy π_θ with discount γ :

$$J(\theta) = \mathbb{E}_{((s_0, o_0, a_0), (s_1, o_1, a_1), \dots) \sim p_\theta} \left[\sum_t \gamma^t R(s^t, a^t) \right] \quad (1)$$

Vision-Language Model. Numerous vision-language models (VLMs) have been developed that can concurrently process visual and language input. These models can broadly be classified into two categories [8]: representation learning models, such as CLIP [41], and generative models, such as Blip-2 [42] and InstructBlip [43]. Following [8], [34], [11], we particularly employ the generative VLMs in the format of $\{\text{vision, text}\} \rightarrow \{\text{text}\}$. Formally, the generative VLMs sample tokens $x^{1:K}$ from $p(x^{1:K} | I, c)$, which are conditioned on the input image I and instruction c . Since original generative VLMs produce natural language outputs, integrating these models into robotic control tasks requires an additional action modeling component, detailed in the subsequent section.

IV. METHOD

Our goal is to develop a learning method that effectively improves the VLA model through online interactions while maintaining computational costs affordable for robotic systems. We start with a Vision-Language-Action (VLA) model fine-tuned on robotic demonstrations. We detail the VLA architectures in Section IV-A and outline the learning pipeline of the iRe-VLA method in Section IV-B.

A. Model Architectures

Our VLA model transforms vision input $o \in \mathcal{O}$ and free-form language instruction $i \in \mathcal{L}$ into low-level robotic action $a \in \mathcal{A}$, represented as $\mathcal{O} \times \mathcal{L} \rightarrow \mathcal{A}$. The model comprises a pre-trained large VLM and a lightweight action head, as illustrated on the left side of Figure 2.

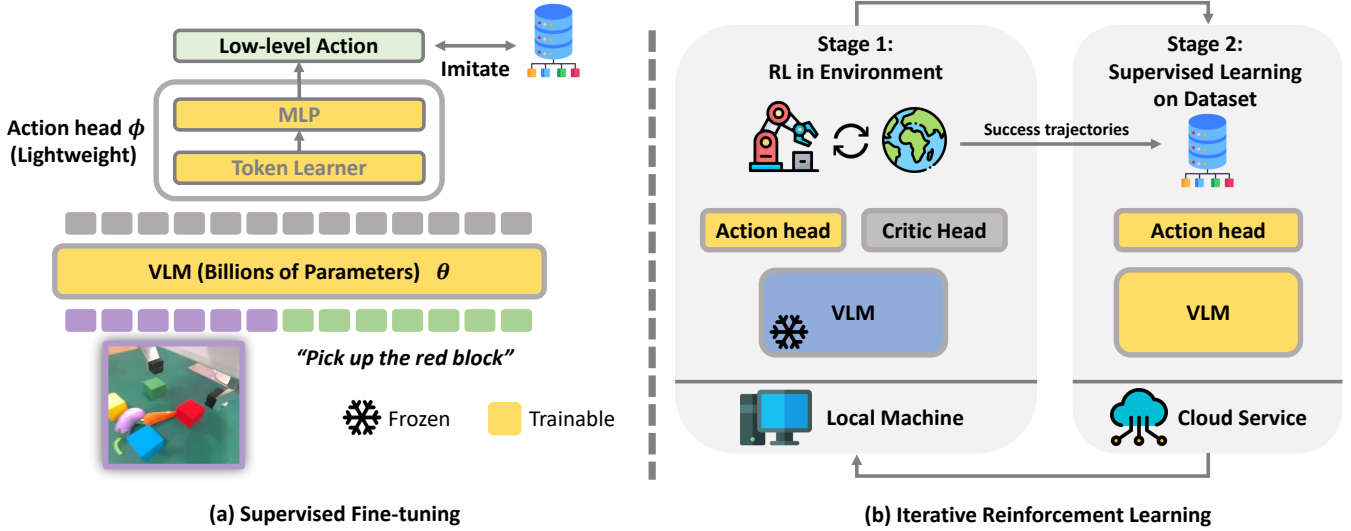


Fig. 2: (a) Our VLA model comprised a pre-trained VLM backbone and lightweight action head. (b) During the finetuning, we iterate between exploration and SL stages to effectively improve the VLA model. The VLM is frozen in the exploration stage to stabilize training and trainable in the SL stage to fully leverage the power of pre-trained VLM.

We utilize the BLIP-2 3B model [42] as our backbone VLM. Since pre-trained VLM output text tokens in language space, an action head is designed to produce low-level control actions. These actions typically include changes in the end-effector’s pose and the gripper’s status. Following the design presented in [11], [34], we replace the VLM’s final fully connected layer with a newly initialized action head. In the action head, a token learner [44] first converts the VLM’s last hidden representation $h \in \mathbb{R}^{m \times d}$ to $h' \in \mathbb{R}^d$. Subsequently, a Multi-Layer-Perceptron (MLP) [45] map h' to the action $a \in \mathbb{R}^{d_a}$, where m and d denote the number of tokens and the embedding dimension of the VLM, respectively, and d_a represents the action dimensions.

Low-Rank Adaptation (LoRA) [46] Our VLA model comprises a large VLM backbone and a lightweight action head. However, fine-tuning the entire model, with its billions of parameters, requires significant computational resources. Furthermore, previous studies [47], [48] suggest that fine-tuning the whole large pre-trained model in limited-data regimens can result in over-fitting. Following the approach described in [47], we utilize the parameter-efficient LoRA method to fine-tune the VLM part. The total trainable parameters consist of the LoRA parameters θ and the action head parameters ϕ .

B. Learning Pipeline

We describe the learning pipeline in this section. First, we supervised fine-tuning the VLA model on robotic datasets (stage 0), then we iterate between online RL (stage 1) and supervised learning (stage 2).

Stage 0: Supervised Learning on Expert Dataset. We first perform standard supervised fine-tuning on the VLA model π_θ with the expert robotic dataset $D_e = \{(o_1, l_1, a_1), (o_2, l_2, a_2), \dots, (o_i, l_i, a_i)\}$. Formally, the learning objective is defined by a Mean Squared Error (MSE)

Algorithm 1 Iterative RL for VLA model (iRe-VLA)

Given: A expert dataset D_e , a supervise fine-tuned VLA model $\pi_{\theta, \phi}^0$ with VLM parameters θ and action head ϕ , **unseen tasks set** $T = \{T_1, \dots, T_n\}$.

- 1: Initialize the online dataset $D_{RL} \leftarrow \emptyset$, copy the weight of $\pi_{\theta, \phi}^0$ to $\pi_{\theta, \phi}^1, \pi_{\theta, \phi}^2$
- 2: **for** T_i **in** $\{T_0, T_1, \dots, T_n\}$ **do**
- 3: **# Stage 1: RL**
- 4: Copy the weight of $\pi_{\theta, \phi}^2$ to $\pi_{\theta, \phi}^1$, initialize a critic head.
- 5: Optimize ϕ with online reinforcement learning until convergence by equation 3.
- 6: Collect successful trajectories x_i into $D_{RL}: D_{RL} = D_{RL} \cup x_i$.
- 7: **# Stage 2: SL**
- 8: Copy the weight of $\pi_{\theta, \phi}^1$ to $\pi_{\theta, \phi}^2$.
- 9: Optimize θ, ϕ with supervised learning on $D_e \cup D_{RL}$ by equation 4.
- 10: **end for**

loss:

$$J^0(\theta, \phi) = \mathbb{E}_{(o, l, a) \sim D_e} [\|\pi_{\theta, \phi}(o, l) - a\|_2^2] \quad (2)$$

After supervised fine-tuning, we obtain the initial VLA model $\pi_{\theta, \phi}^0$. The performance of $\pi_{\theta, \phi}^0$ is highly correlated to the scale and quality of the expert dataset D_e . Then we start to improve the $\pi_{\theta, \phi}^0$ through online RL.

Stage 1: Online RL with Frozen VLM. The SFT model, $\pi_{\theta, \phi}^0$, may not achieve optimal performance for new tasks. However, it serves as a valuable starting point since it has been trained on a variety of tasks from the robotic dataset. To enhance the performance of the SFT policy, we utilize online reinforcement learning (RL). In the RL process, we introduce

a critic head that mirrors the structure of the action head, but with the output dimension set to one. To prevent model collapse and accelerate the learning process, we freeze the VLM parameters, θ , during this phase. Consequently, only the parameters of the action head, ϕ , are optimized:

$$J^1(\phi) = \mathbb{E}_{((s_0, o_0, a_0), (s_1, o_1, a_1), \dots) \sim p_\phi} \left[\sum_t \gamma^t R(o^t, a^t) \right] \quad (3)$$

After online RL, the robot may discover new trajectories x_i to solve new tasks. Then we collected these success trajectories into an online dataset $D_{RL} = D_{RL} \cup x_i$

Stage 2: Supervised Learning on Both Expert and Online-collected Data. In Stage 1, while the agent conducts RL on new tasks, it risks forgetting previously learned tasks. Hence, in Stage 2, we supervise the whole model using both the newly collected online data D_{RL} and the original expert dataset D_e to mitigate catastrophic forgetting [49]. Formally, the objective can be written as:

$$J^2(\theta, \phi) = \mathbb{E}_{(o, l, a) \sim D_e \cup D_{RL}} \left[\|\pi_{\theta, \phi}(o, l) - a\|_2^2 \right] \quad (4)$$

Iterate between Stage 1 and Stage 2. As previously noted, the agent in Stage 1 explores novel solutions for new tasks, while in Stage 2, it imitates all available success trajectories. By alternating between Stages 1 and 2, large VLA models progressively address a broader range of tasks while also preventing catastrophic forgetting on seen tasks. Furthermore, as suggested in previous works [50], [13], the VLA model could become more generalizable by imitating a wider range of tasks. The whole pipeline is outlined in Algorithm 1.

V. EXPERIMENTS

In this section, we perform tense experiments in two simulated benchmarks Metaworld and FrankaKitchen, and real-world panda manipulation tasks to verify the effectiveness of our iRe-VLA framework. We aim to answer the following questions:

- Why do we adopt a two-stage iterative RL process instead of standard RL?
- Can iRe-VLA stabilize the training process and effectively improve the VLA model in both expert tasks and unseen tasks?
- Can iRe-VLA lead to better generalization of the VLA model?

A. Experiment Setups

We perform experiments in three domains: Metaworld [21], Franka Kitchen [22], and real-world panda manipulation, as illustrated in Figure 3. Notably, we use a **single text-conditioned VLA model** to solve all tasks in a domain. Each domain involves tasks categorized into three groups: expert tasks observed in the demonstration datasets, RL-trained tasks enhanced by online RL, and hold-out tasks that are unseen in prior training. Initially, we conducted supervised fine-tuning on the VLA model using expert datasets. Subsequently, we improve the performance of the

VLA model in second-category new tasks through online RL. Lastly, the third-category tasks are employed to evaluate the generalization capabilities of the trained VLA policy.

In the Metaworld domain, the expert dataset contains 25 tasks each with 50 trajectories. The second and third category introduces novel tasks featuring variations in object shape, color, and position. In the Franka kitchen domain, we follow the setting in [47], the expert dataset contains 5 tasks while the tasks in the second and third categories encompass unseen changes in object appearance and position. As for real-world tasks, we collect 2,000 trajectories through teleoperation and script for picking (grasp), placing, button-press, cable-route, and drawer-open. The unseen tasks of real-world experiments include picking up unseen objects.

B. Why do we adopt two-stage iterative optimization?

Stabilizing Training Process. We observed that directly fine-tuning the large VLA model using standard reinforcement learning (RL) algorithms can be unstable and lead to performance drops. As shown in Figure 3, we observe performance drops in four out of five tasks with sparse reward in the Metaworld benchmark. This phenomenon was also observed in previous research [18], which encountered similar instability issues with transformer-based RL policies and had to modify transformer blocks to prevent collapse. However, these modifications are not compatible with pre-trained VLMs, instead, we freeze the VLM during the RL stage to prevent collapse.

Managing the Model Training Burden. Fully fine-tuning the VLA model with billions of parameters exceeds the computational capability of most local machines, while complete deployment on a remote server introduces parameter transmission issues and reduces the control frequency. Our two-stage iRe-VLA framework addresses these challenges by distributing the computational load. In the first RL stage, iRe-VLA freezes the upper-layer VLM and only adapts the lightweight action head, thus keeping computational demands affordable on the local machine. The second stage of optimization is then delegated to remote services that can handle larger computational loads. For instance, in our real-world experiments (see Section V-D), we conducted the RL process locally using a single NVIDIA 4090 card and performed the second stage on remote servers equipped with 4 NVIDIA A100 cards.

C. Simulated Manipulation Experiments

We initially conducted experiments in simulated Metaworld and Franka Kitchen benchmark, where the VLA model is first supervised on 25 tasks and 5 tasks respectively. VLA model can provide an effective starting point for RL tasks, accelerating the RL process compared to the learn-from-scratch approach, as demonstrated in Figure 4. Subsequently, we perform the iRe-VLA method to learn RL tasks one by one, which continuously improves the VLA model. We compare our method with the standard PPO algorithm [51]. To ensure a fair comparison, we also performed PPO task by

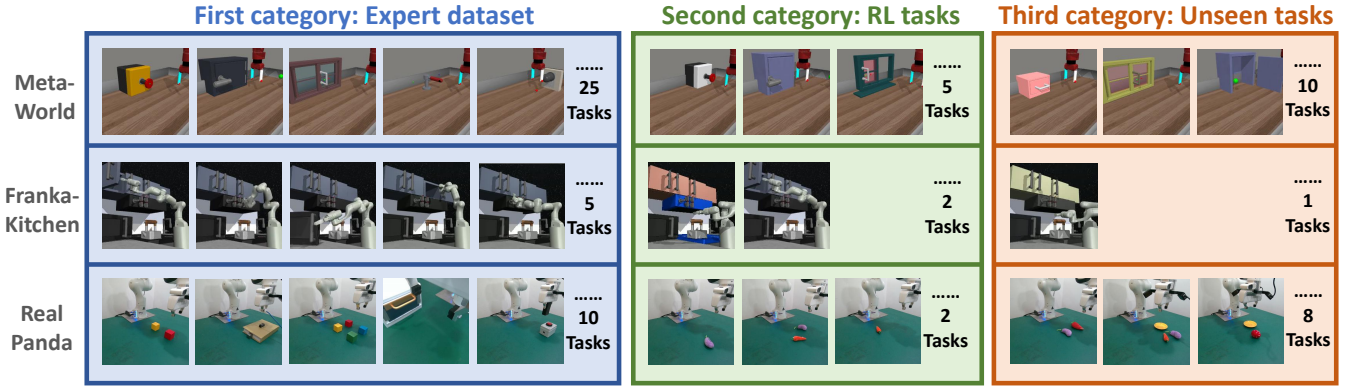


Fig. 3: We perform experiments in three domains. Each domain encompasses three categories: tasks observed in the expert dataset, new tasks utilizing reinforcement learning, and hold-out unseen tasks. The tasks vary by required skills, as well as the shapes and appearances of objects. The initial positions of objects in each task are randomized in every episode.

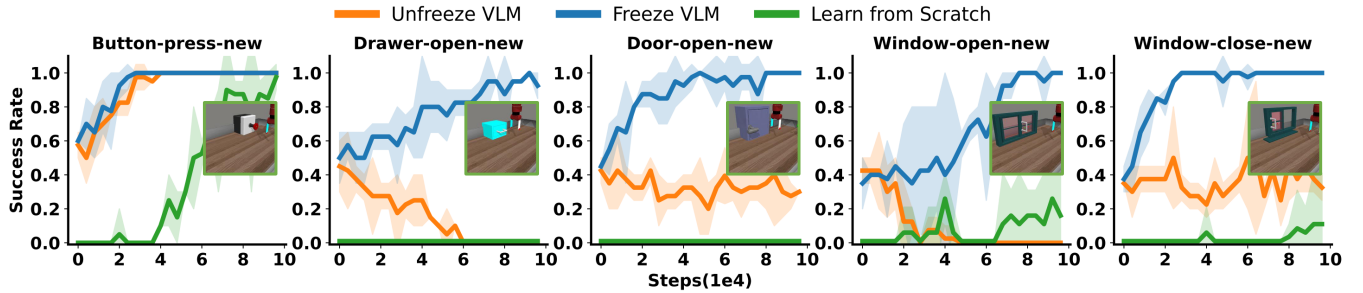


Fig. 4: Reinforcement Learning process in new tasks. SFT policy can serve as a good starting point in new RL tasks compared to the learn-from-scratch policy. We also observed that fully fine-tuning VLA models can lead to performance degradation (orange lines) while freezing the VLM part can avoid collapses.

Metaworld	Original 25 tasks	Button- Press-new	Drawer- Open-new	Door- Open-new	Window- Open-new	Window- Close-new	Unseen 10 tasks
SFT Policy	0.83	0.56	0.48	0.40	0.32	0.28	0.51
PPO-Replay	0.69	0.80	0.24	0.32	0.04	0.36	0.39
iRe-VLA(Ours)	0.83	1.00	0.84	0.84	0.80	0.96	0.80

Franka Kitchen	Knob-on	Light-on	Microwave -open	Slide-door -open	Left-door -open	Slide-door -open-red	Slide-door -open-yellow
SFT Policy	0.84	0.96	0.70	0.86	0.43	0.46	0.98
PPO-Replay	0.48	0.64	0.35	0.96	0.12	0.30	0.64
iRe-VLA(Ours)	0.90	0.98	0.82	0.99	0.83	0.99	1.00

TABLE I: Success rates on Metaworld and Franka-kitchen benchmark with three categories of tasks (expert tasks in blue, RL-trained tasks in green, and unseen tasks in red). Standard online RL algorithms result in performance even worse than SFT policy, while iRe-VLA improves performance in three categories of tasks.

task and adopted the same expert data replay strategy after each task, namely PPO-Replay.

Analysis. The results are presented in Table I. Standard PPO algorithms often exhibit instability when introduced to RL tasks, as depicted in Figure 4. This instability not only affects performance in RL tasks but also degrades performance in previously learned tasks, even with experience replay. This decline is likely due to noisy RL gradients that adversely affect the pre-trained representations within the VLA model. In contrast, our two-stage iRe-VLA method stabilizes the RL process and effectively enhances task performance across

both seen and unseen tasks. The advantage of the iRe-VLA method can be reflected in three aspects:

(1) **Improved Performance in Original Tasks.** We can continue to improve performance in seen expert tasks through online interaction. For instance, in the Franka-kitchen benchmark, the supervised VLA model achieved a modest success rate in the expert task *left-door-open* due to limited demonstrations. Our iRe-VLA method improves the success rate of this task from 0.43 to 0.83.

(2) **Improved Performance in RL Tasks.** It is crucial for intelligent agents to adapt to tasks excluded in expert data

autonomously. We explored various RL tasks (as detailed in the second column of Figure 3) and applied our iterative RL algorithm to address these tasks. As indicated in Table 3, our iRe-VLA method successfully tackled new tasks in each domain without catastrophic forgetting [49].

(3) Improved Generalization in Unseen Tasks. In addition to the enhanced performance in RL-trained tasks through online iterations, we also observed increased success rates in unseen tasks, indicating better generalization ability. As the agent tackles an increasing variety of tasks automatically, its generalization ability correspondingly strengthens. For example, after mastering four types of window tasks in Metaworld, the agent effectively generalized to windows of unseen colors and shapes.

Ablation Study. In the iRe-VLA method, the whole VLM is trainable in the second supervised learning stage. We conducted ablation studies by freezing the VLM in both stages, namely iRe-VLA-freeze. In this way, online iteration data can not affect the VLM latent. The outcomes, depicted in Figure 5, suggest that permanently freezing the VLM leads to a reduction in performance. This could be attributed to the action head’s limited expressiveness compared to the full VLA model. Additionally, online robotic action data could enhance the representations in the upper-layer VLM, thereby augmenting the VLA model’s generalizability in unseen tasks, while freezing VLM in both stages can not improve the VLM representation.

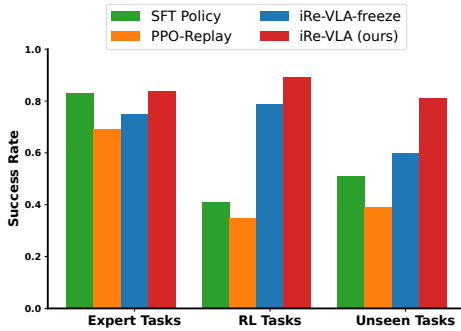


Fig. 5: Ablations. Freezing VLM all the time leads to performance drops.

D. Real-world Manipulation Experiments

Experiment Setups. Our real-world experiment follows the set ups described in SERL [52], [53], a useful software suite for the real-world RL. We first train a VLA model on 2,000 human-collected expert data across various task categories, including pick (grasp), place, button-press, cable-route, and drawer operations.

We notice that the learned VLA model shows a certainty success rate on unseen objects thanks to the generalization ability of the VLA model. Then we adopt online RL to further increase the success rate on unseen objects. We implemented several key design choices to enhance sample efficiency and ensure computational affordability within the context of large Vision-Language-Action (VLA) models. To improve sample efficiency, we adopted the SACfD algorithm [54], [55]. Specifically, when introduced to a new task, we

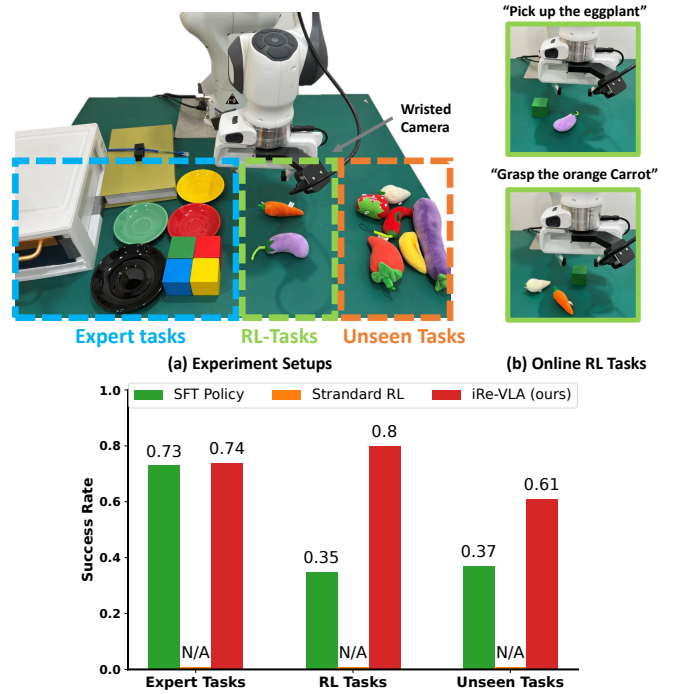


Fig. 6: Real-world experiments with panda arm. We did not report the standard RL results in real-world tasks since directly fine-tuning the entire VLA model exceeded the computational capabilities of our local machine.

initially utilize zero-shot transferred VLA models to collect a demonstration buffer containing 20 successful trajectories. During training, we sample 50% transitions from the demonstration buffer and 50% from the online buffer, as outlined in [52]. To manage computational costs, each image observation is processed by the VLM only once, and the resulting latent output is stored in the buffer. Subsequently, we implement the SACfD algorithm in this latent space.

Results. The expert pick demonstrations were limited to blocks of four colors, and we extended the online RL to objects with irregular shapes, such as eggplants and carrots. The real-world RL training process for each new task costs around one hour, similar to time costs in SERL [52]. The success rates before and after RL process are shown in Figure 6, our iRe-VLA pipeline increased the success rate for picking eggplants or carrots from 0.35 to 0.80. Moreover, the success rates for the original tasks remained stable, and the picking success rate for unseen objects also improved from 0.37 to 0.61.

VI. CONCLUSION AND LIMITATION

In this paper, we explore ways to further enhance the VLA model through online reinforcement learning. Fine-tuning large VLA models presents several challenges, but our proposed iRe-VLA methods stabilize the training process and significantly reduce computational demands. Experiments on both simulated and real-world manipulation tasks confirm the effectiveness of iRe-VLA. A potential limitation is that it can only improve skills within seen types and cannot learn entirely new skills under sparse-reward online RL conditions.

REFERENCES

- [1] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, “Training language models to follow instructions with human feedback,” *Advances in neural information processing systems*, vol. 35, pp. 27730–27744, 2022.
- [2] A. Glaese, N. McAleese, Maja, J. Aslanides, V. Firoiu, T. Ewalds, M. Rauh, L. Weidinger, M. Chadwick, P. Thacker, *et al.*, “Improving alignment of dialogue agents via targeted human judgements,” *arXiv preprint arXiv:2209.14375*, 2022.
- [3] R. Thoppilan, D. De Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H.-T. Cheng, A. Jin, T. Bos, L. Baker, Y. Du, *et al.*, “Lamda: Language models for dialog applications,” *arXiv preprint arXiv:2201.08239*, 2022.
- [4] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [5] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [6] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, *et al.*, “Inner monologue: Embodied reasoning through planning with language models,” *arXiv preprint arXiv:2207.05608*, 2022.
- [7] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, *et al.*, “Rt-1: Robotics transformer for real-world control at scale,” *arXiv preprint arXiv:2212.06817*, 2022.
- [8] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv preprint arXiv:2307.15818*, 2023.
- [9] Y. Ma, Z. Song, Y. Zhuang, J. Hao, and I. King, “A survey on vision-language-action models for embodied ai,” *arXiv preprint arXiv:2405.14093*, 2024.
- [10] J. Zhang, Y. Guo, X. Chen, Y.-J. Wang, Y. Hu, C. Shi, and J. Chen, “Hirt: Enhancing robotic control with hierarchical robot transformers,” in *8th Annual Conference on Robot Learning*.
- [11] X. Li, M. Liu, H. Zhang, C. Yu, J. Xu, H. Wu, C. Cheang, Y. Jing, W. Zhang, H. Liu, *et al.*, “Vision-language foundation models as effective robot imitators,” *arXiv preprint arXiv:2311.01378*, 2023.
- [12] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in neural information processing systems*, vol. 35, pp. 24824–24837, 2022.
- [13] A. Padalkar, A. Pooley, A. Jain, A. Bewley, A. Herzog, A. Irpan, A. Khazatsky, A. Rai, A. Singh, A. Brohan, *et al.*, “Open x-embodiment: Robotic learning datasets and rt-x models,” *arXiv preprint arXiv:2310.08864*, 2023.
- [14] S. Belkale, Y. Cui, and D. Sadigh, “Data quality in imitation learning,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [15] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative q-learning for offline reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1179–1191, 2020.
- [16] F. Liu *et al.*, “Learning to summarize from human feedback,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.
- [17] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” *Advances in neural information processing systems*, vol. 30, 2017.
- [18] E. Parisotto, F. Song, J. Rae, R. Pascanu, C. Gulcehre, S. Jayakumar, M. Jaderberg, R. L. Kaufman, A. Clark, S. Noury, *et al.*, “Stabilizing transformers for reinforcement learning,” in *International conference on machine learning*. PMLR, 2020, pp. 7487–7498.
- [19] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski, *et al.*, “What matters for on-policy deep actor-critic methods? a large-scale study,” in *International conference on learning representations*, 2020.
- [20] K. Ota, D. K. Jha, and A. Kanezaki, “Training larger networks for deep reinforcement learning,” *arXiv preprint arXiv:2102.07920*, 2021.
- [21] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, “Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning,” in *Conference on robot learning*. PMLR, 2020, pp. 1094–1100.
- [22] A. Gupta, V. Kumar, C. Lynch, S. Levine, and K. Hausman, “Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning,” *arXiv preprint arXiv:1910.11956*, 2019.
- [23] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, “Eureka: Human-level reward design via coding large language models,” *arXiv preprint arXiv:2310.12931*, 2023.
- [24] L. Fan, G. Wang, Y. Jiang, A. Mandlekar, Y. Yang, H. Zhu, A. Tang, D.-A. Huang, Y. Zhu, and A. Anandkumar, “Minedojo: Building open-ended embodied agents with internet-scale knowledge,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 18343–18362, 2022.
- [25] A. Adeniji, A. Xie, C. Sferrazza, Y. Seo, S. James, and P. Abbeel, “Language reward modulation for pretraining reinforcement learning,” *arXiv preprint arXiv:2308.12270*, 2023.
- [26] J. Lin, Y. Du, O. Watkins, D. Hafner, P. Abbeel, D. Klein, and A. Dragan, “Learning to model the world with language,” *arXiv preprint arXiv:2308.01399*, 2023.
- [27] A. W. Hanjje, V. Y. Zhong, and K. Narasimhan, “Grounding language to entities and dynamics for generalization in reinforcement learning,” in *International Conference on Machine Learning*. PMLR, 2021, pp. 4051–4062.
- [28] A. Zeng, A. Wong, S. Welker, K. Choromanski, F. Tombari, A. Purohit, M. Ryoo, V. Sindhwani, J. Lee, V. Vanhoucke, *et al.*, “Socratic models: Composing zero-shot multimodal reasoning with language,” *arXiv preprint arXiv:2204.00598*, 2022.
- [29] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, *et al.*, “Palm-e: An embodied multimodal language model,” *arXiv preprint arXiv:2303.03378*, 2023.
- [30] Y. Guo, Y.-J. Wang, L. Zha, Z. Jiang, and J. Chen, “Doremi: Grounding language model by detecting and recovering from plan-execution misalignment,” *arXiv preprint arXiv:2307.00329*, 2023.
- [31] I. Dasgupta, C. Kaeser-Chen, K. Marino, A. Ahuja, S. Babayan, F. Hill, and R. Fergus, “Collaborating with language models for embodied reasoning,” *arXiv preprint arXiv:2302.00763*, 2023.
- [32] Y.-J. Wang, B. Zhang, J. Chen, and K. Sreenath, “Prompt a robot to walk with large language models,” *arXiv preprint arXiv:2309.09969*, 2023.
- [33] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” *arXiv preprint arXiv:2209.07753*, 2022.
- [34] W. Chen, O. Mees, A. Kumar, and S. Levine, “Vision-language models provide promptable representations for reinforcement learning,” *arXiv preprint arXiv:2402.02651*, 2024.
- [35] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano, “Learning to summarize with human feedback,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 3008–3021, 2020.
- [36] R. Ramamurthy, P. Ammanabrolu, K. Brantley, J. Hessel, R. Sifa, C. Bauckhage, H. Hajishirzi, and Y. Choi, “Is reinforcement learning (not) for natural language processing: Benchmarks, baselines, and building blocks for natural language policy optimization,” *arXiv preprint arXiv:2210.01241*, 2022.
- [37] S. Levine, A. Kumar, G. Tucker, and J. Fu, “Offline reinforcement learning: Tutorial, review, and perspectives on open problems,” *arXiv preprint arXiv:2005.01643*, 2020.
- [38] T. Carta, C. Romac, T. Wolf, S. Lamprier, O. Sigaud, and P.-Y. Oudeyer, “Grounding large language models in interactive environments with online reinforcement learning,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 3676–3713.
- [39] A. Szot, M. Schwarzer, H. Agrawal, B. Mazouze, R. Metcalf, W. Talbott, N. Mackraz, R. D. Hjelm, and A. T. Toshev, “Large language models as generalizable policies for embodied tasks,” in *The Twelfth International Conference on Learning Representations*, 2023.
- [40] Y. Zhai, H. Bai, Z. Lin, J. Pan, S. Tong, Y. Zhou, A. Suhr, S. Xie, Y. LeCun, Y. Ma, *et al.*, “Fine-tuning large vision-language models as decision-making agents via reinforcement learning,” *arXiv preprint arXiv:2405.10292*, 2024.
- [41] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, *et al.*, “Learning transferable

- visual models from natural language supervision,” in *International conference on machine learning*. PMLR, 2021, pp. 8748–8763.
- [42] J. Li, D. Li, S. Savarese, and S. Hoi, “Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” in *International conference on machine learning*. PMLR, 2023, pp. 19 730–19 742.
 - [43] W. Dai, J. Li, D. Li, A. M. H. Tiong, J. Zhao, W. Wang, B. Li, P. N. Fung, and S. Hoi, “Instructblip: Towards general-purpose vision-language models with instruction tuning,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
 - [44] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh, “Set transformer: A framework for attention-based permutation-invariant neural networks,” in *International conference on machine learning*. PMLR, 2019, pp. 3744–3753.
 - [45] M. Riedmiller and A. Lerner, “Multi layer perceptron,” *Machine Learning Lab Special Lecture, University of Freiburg*, vol. 24, 2014.
 - [46] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.
 - [47] Z. Liu, J. Zhang, K. Asadi, Y. Liu, D. Zhao, S. Sabach, and R. Fakoor, “Tail: Task-specific adapters for imitation learning with large pre-trained models,” *arXiv preprint arXiv:2310.05905*, 2023.
 - [48] K. Bousmalis, G. Vezzani, D. Rao, C. Devin, A. X. Lee, M. Bauza, T. Davchev, Y. Zhou, A. Gupta, A. Raju, *et al.*, “Robocat: A self-improving foundation agent for robotic manipulation,” *arXiv preprint arXiv:2306.11706*, 2023.
 - [49] M. McCloskey and N. J. Cohen, “Catastrophic interference in connectionist networks: The sequential learning problem,” in *Psychology of learning and motivation*. Elsevier, 1989, vol. 24, pp. 109–165.
 - [50] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, *et al.*, “Octo: An open-source generalist robot policy,” 2023.
 - [51] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
 - [52] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine, “Serl: A software suite for sample-efficient robotic reinforcement learning,” *arXiv preprint arXiv:2401.16013*, 2024.
 - [53] J. Luo, C. Xu, F. Liu, L. Tan, Z. Lin, J. Wu, P. Abbeel, and S. Levine, “Fmb: a functional manipulation benchmark for generalizable robotic learning,” *arXiv preprint arXiv:2401.08553*, 2024.
 - [54] M. Vecerik, T. Hester, J. Scholz, F. Wang, O. Pietquin, B. Piot, N. Heess, T. Rothörl, T. Lampe, and M. Riedmiller, “Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards,” *arXiv preprint arXiv:1707.08817*, 2017.
 - [55] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *International conference on machine learning*. PMLR, 2018, pp. 1861–1870.